Chapter 3 Transport Layer

Transport-layer services

Transport vs. Network layer

Network layer (between hosts): 通過路由與轉發將資送到目的裝置

Transport Layer (between processes): 提供 Port Number 來對應 Application

Two principal Internet transport protocols

TCP: Transmission Control Protocol

1. Reliable, in-order delivery
2. Congestion control
3. Flow control
4. Connection setup

UDP: User Datagram Protocol

1. unreliable, unordered delivery
2. No-frills (樸實無華) extension of "best-effort" IP

Services not available in both:

Delay & bandwidth: Because of the packet switching

Multiplexing and demultiplexing

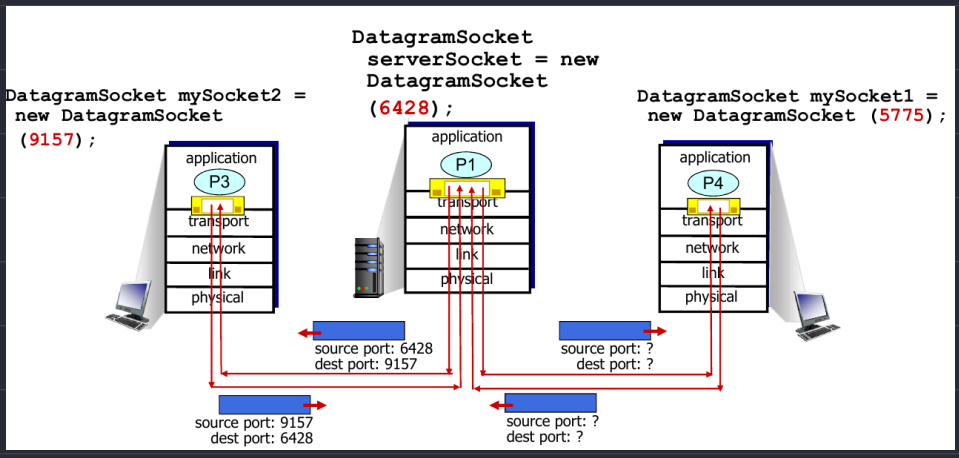
Port Number is used in transport layer to MUX/DEMUX to different app in same host

How demultiplexing works

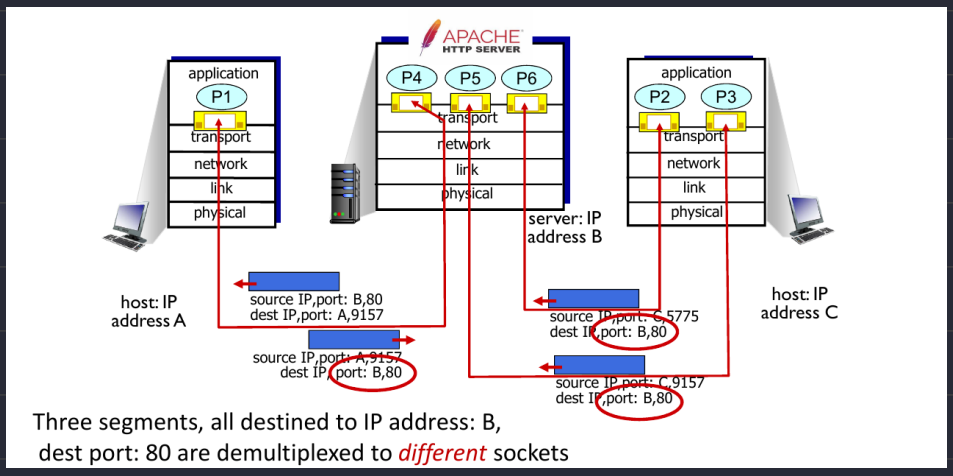
Host uses IP addresses & Port numbers in packet to direct to socket

Connectionless demultiplexing

1. UDP ⟹ Use DEST IP & DEST PORT to identify a socket



2. TCP ⟹ Use SRC IP & SRC PORT & DEST IP &DEST PORT to identify a socket



Summary

1. Multiplexing, Demultiplexing based on segment, datagram header field values
2. Multiplexing / Demultiplexing jappened at all layers

Connectionless transport: UDP

UDP: User Datagram Protocol

"best effort" service: 但是這裡是指沒有提供保證的服務 (e.g. lost, out of order)

“no frills,” “bare bones” Internet transport protocol

Connectionless

No handshaking between UDP sender, receiver (e.g. Send data even server is down)

Each UDP segment handled independently of others

Why is there a UDP?

No connection establishment → Reduce RTT

No connection state at sender and reviever → Simple

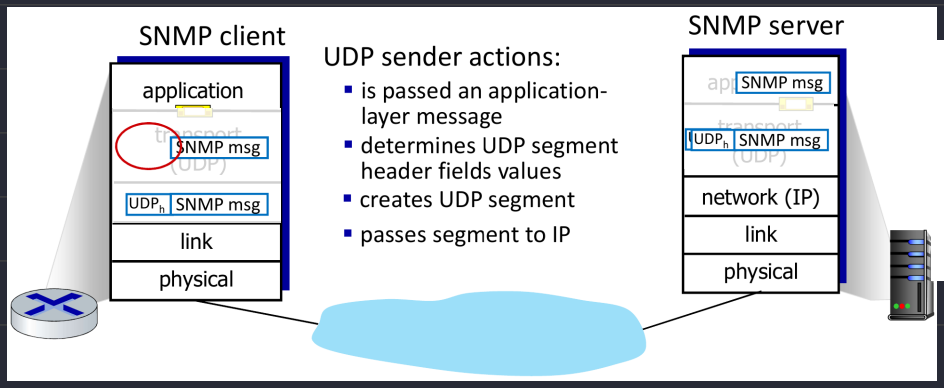
Small header size → 8 byte instead of TCP 20 byte

No congestion control → Always be fast

Usage

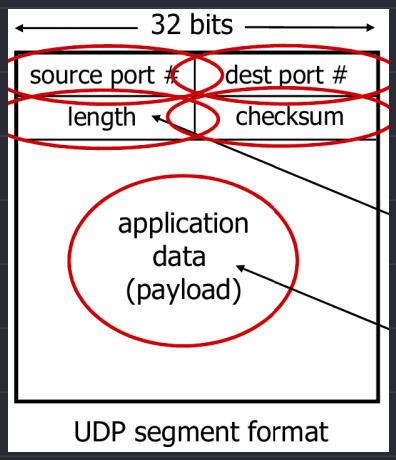
Streaming multimedia apps (loss tolerant, rate sensitive)DNS, HTTP/3, SNMP

UDP: Transport Layer Actions



checksum failed: 不會重傳，只會丟掉

UDP segment header



length: represent in byte

data is from application layer

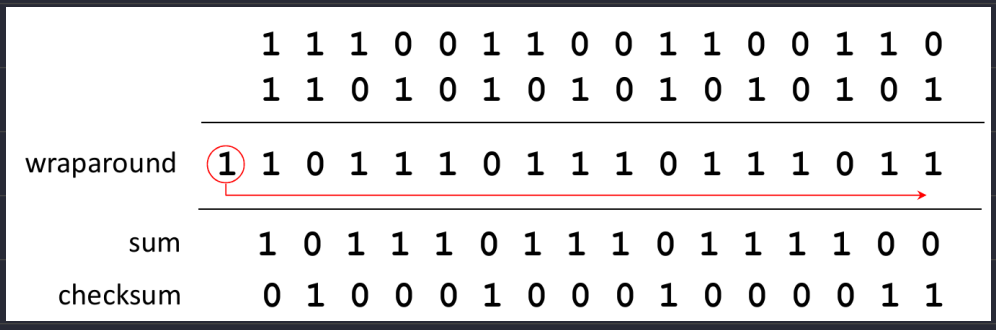
UDP checksum

Internet checksum

Assume that data sent is 1, 5, 6 then checksum 1+5+6 = 12, if sum of data isn't

match the checksum value, means that error occur

→ Checksum can't detect evey error, e.g. data and checksum both been changed



Summary

No Frills (沒有額外的裝飾) Protocol

Segments may be lost or delivered out of order

Best Effort Services "send and hope for the best"

No setup / handshaking needed (RTT Reduced)

Can function in a compromised network

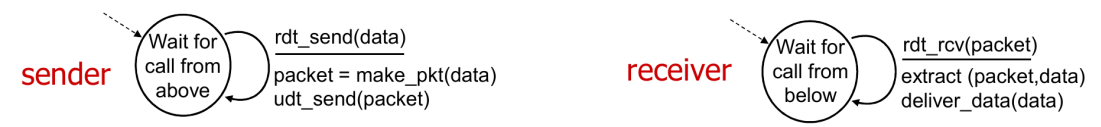
Simple checksum for the reilability

HTTP3 used UDP to transfer data (faster)

Principles of reliable data transfer (RDT)

傳輸方、接收方可以藉由有限狀態機紀錄傳輸資料的狀態，並且無法得知彼此的狀態

rdt1.0: reliable transfer over a reliable channel(a.k.a 幾乎沒有保護)



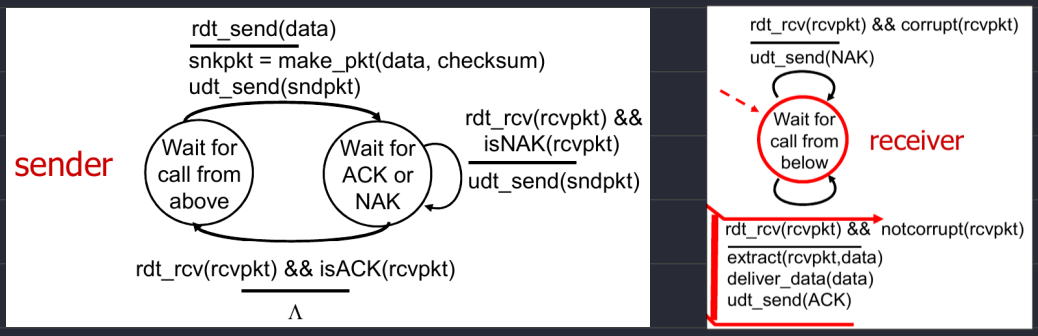
rdt2.0: channel with bit errors (a.k.a 資料失真保護)

underlying channel may flip bits in packet: Use checksum to detect bit error

stop and wait: Sender wait for reciever to send back ACK / NACK, then send next

(Sender need to re-transmit data if NACK recieved)

FSM (Finite State Machine) Chart



rdt2.1: sender, handling garbled (亂碼) ACK/NAKs (a.k.a 資料+ACK失真保護)

ACK / NACK signal might be corrupted by the transmittion

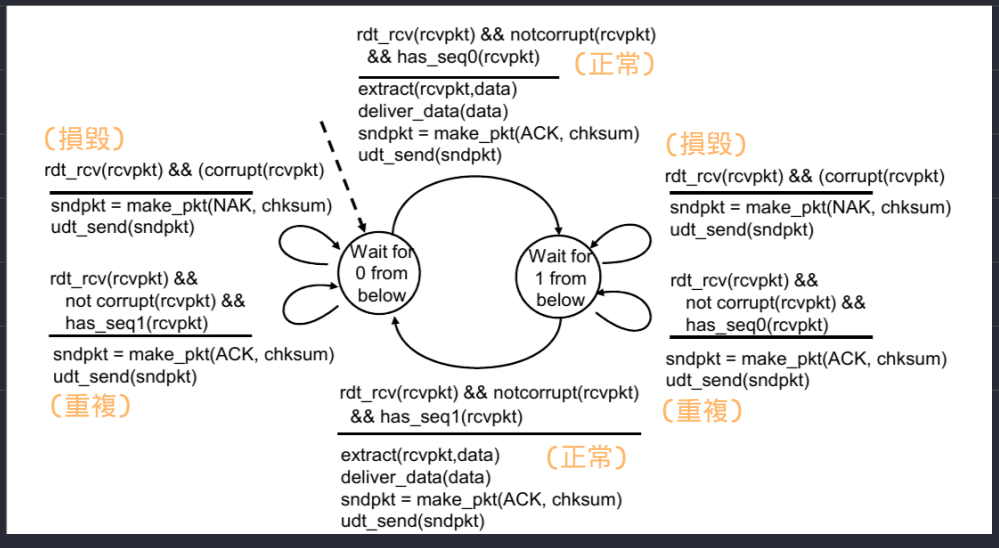
Sender

1. Check if the ACK / NACK corrupted
2. Remember the excepted packet seq number 0 or 1

Receiver

1. Use seq(0~1) to identify whether the packet is duplicated or new
2. Cannot know if the ACK/NACK signal be responsed correctly or not

FSM (Finite State Machine) Chart

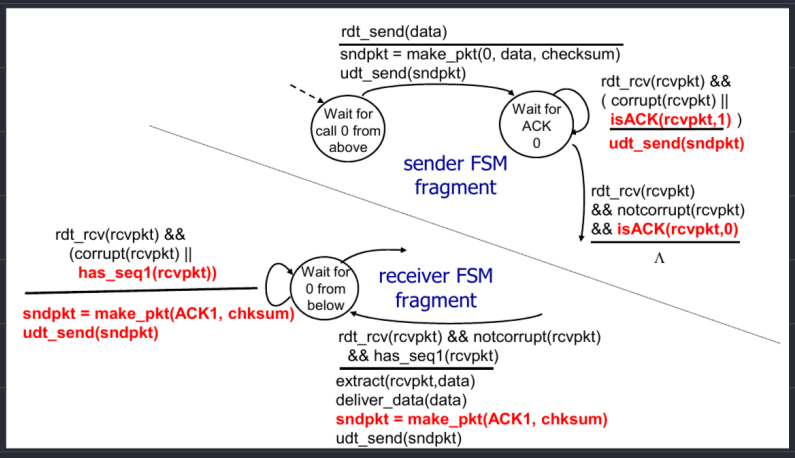


rdt2.2: a NAK-free protocol

Use only ACK to represent if the data be transmitted correctly or not

RDT 2.1	RDT 2.2
ACK	ACK (Current seq)
NACK	ACK (Previous seq)

TCP use this mechanism to be NAK-free

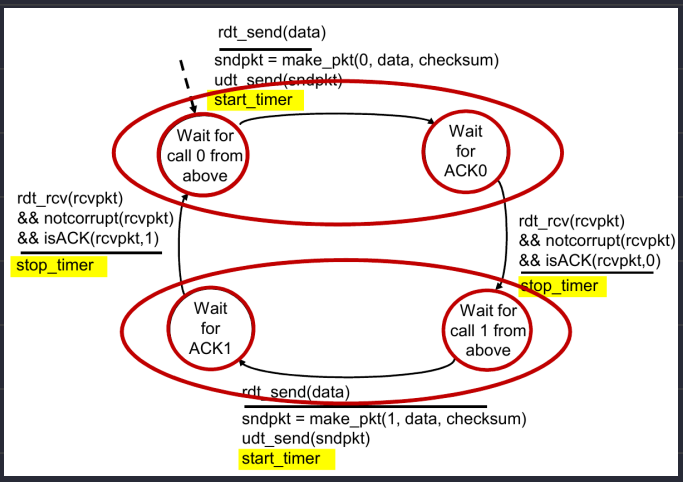


rdt3.0: channels with errors and loss

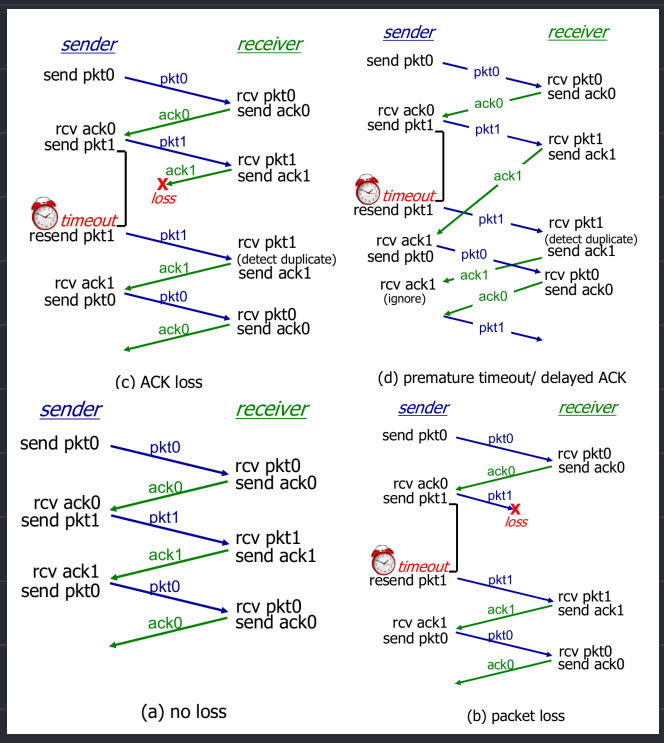
Q: Packet lost in rdt2.0 will lead to wait-forever sceneario

A: Sender waits resonable amount of time for ACK, resend data if timeout

(Resend packet problem has been solved in rdt2.0 with seq method)



rdt3.0 FMS



rdt3.0 in action

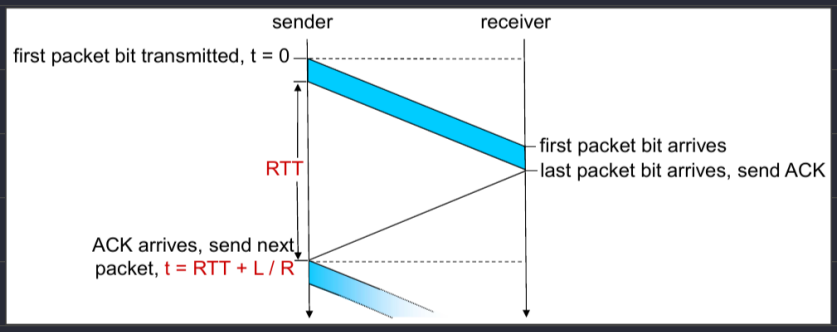
Performance of rdt3.0 (stop-and-wait)

U_{Sender} = utilization : fraction of time sender busy sending

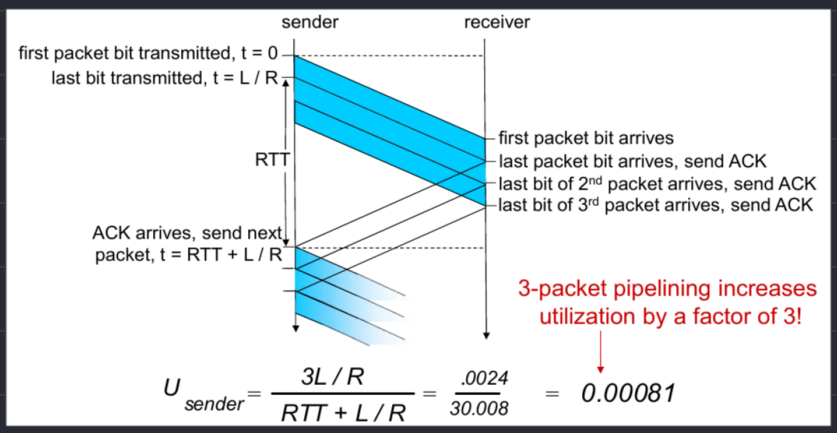
Assume: $D_{trans} = \frac{L}{R} = \frac{8000bits}{10^9 bits/sec} = 8ms$

$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$

... Even though the high bandwidth, usage still limited by high RTT (delay)



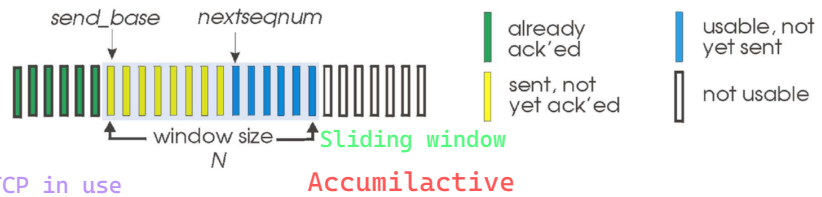
rdt3.0: pipelined protocols operation



Go-Back-N Sender

收到 ACK N 後從 N+1 開始傳

- sender: “window” of up to N, consecutive transmitted but unACKed pkts
  - k-bit seq # in pkt header



Sliding window

TCP in use

Accumilactive

- *cumulative ACK:* ACK(*n*): ACKs all packets up to, including seq # *n*
  - on receiving ACK(*n*): move window forward to begin at *n+1*
- timer for oldest in-flight packet
- *timeout(n):* retransmit packet n and all higher seq # packets in window

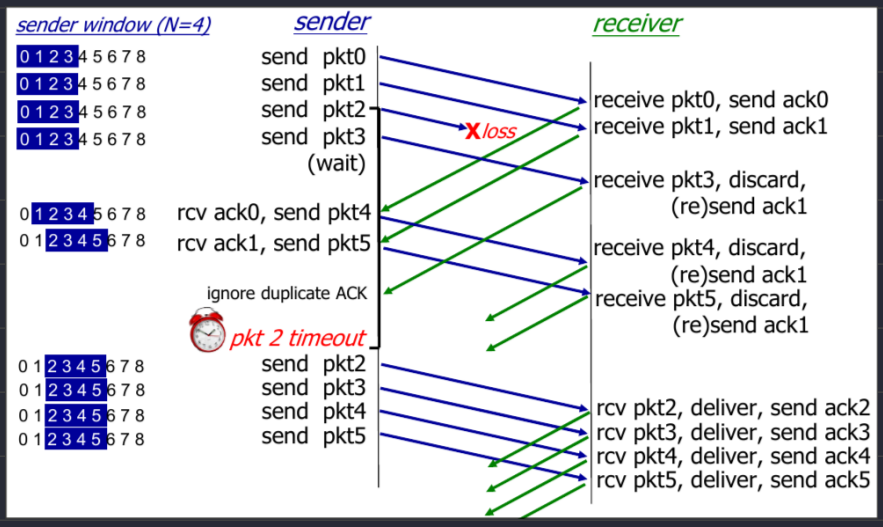
要重新傳送就回到上次 timeout 的第 N 個 pakcet 之後，重送其後面的所有 packet

即使 packet 21 已經正確收到，但是 pakcet 20 沒有，還是回傳 Ack(19)，重送 20 ~ n

Go-Back-N Receiver

- ACK-only: always send ACK for correctly-received packet so far, with highest *in-order* seq #
  - may generate duplicate ACKs
  - need only remember `rcv_base`
- on receipt of out-of-order packet:
  - discard (don’t buffer): *no receiver buffering!*
  - re-ACK pkt with highest in-order seq #

Receiver view of sequence number space:

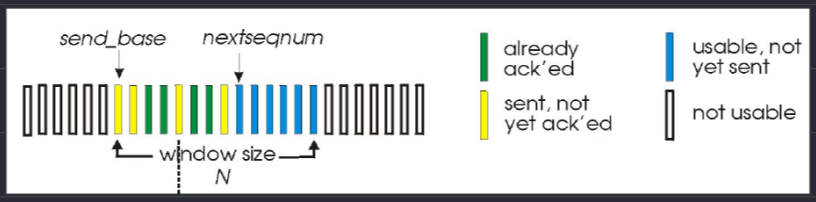


Selective repeat

Receiver:

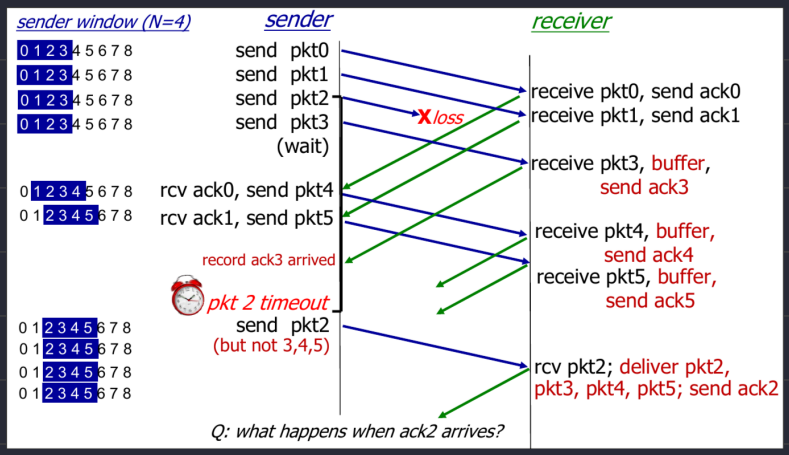
1. Individually acknowledges all correctly received packets
2. Use buffer to store data in case it's out-of-order
3. For packet that is in [rcvbase-N : rcvbase-1], send ACK(N)

Sender:



1. Send next yellow packet in window
2. Mark ACKed packet as already sent
3. Retransmits individually for unACKed and times-out packets
4. Move window to the smallest nmuber from NACK()

Selective Repeat in action

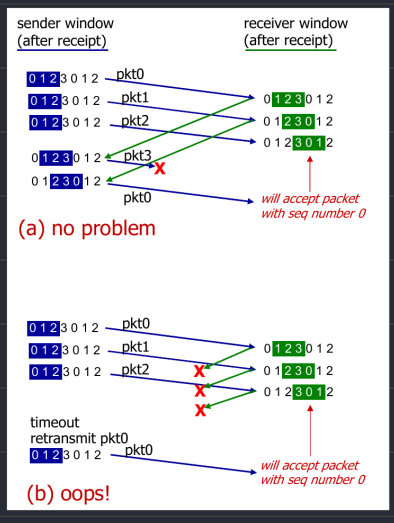


A dilemma in selective repeat

假設 window size = 4, seq = 0 ~ 3 時

Sender send_ptk(0~3) 但是都沒有收到 ACK 所以再重傳

此時重傳的 seq=0 會被放到後面 buffer 導致錯誤的資料接收



解法：讓 seq_number ≥ 2 * window_size

Connection-oriented transport: TCP

TCP: overview

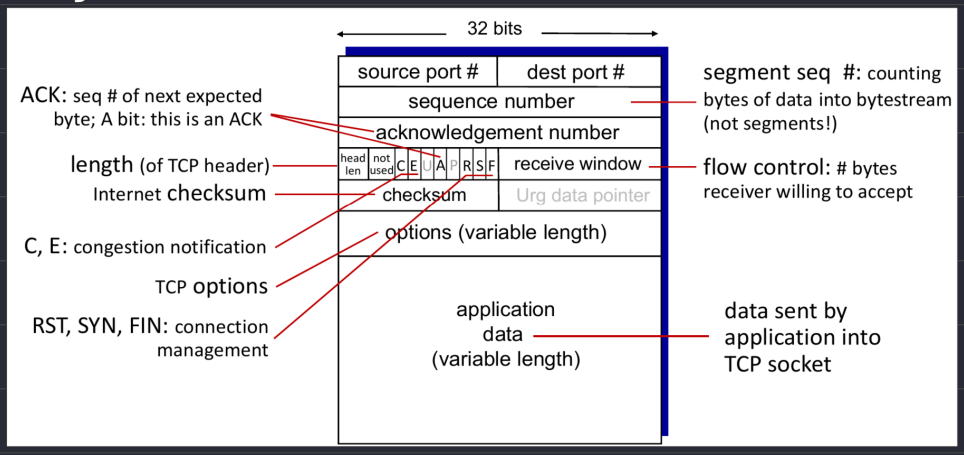
Point to point: One sender, one reviecer

Reliable, in-order byte stream: no message boundaries

Full duplex data Connection oriented Flow controlled

Cumulative (累積) ACKs: ACK(N) 表示 N與之前的pkct都有收到了

TCP segment structure

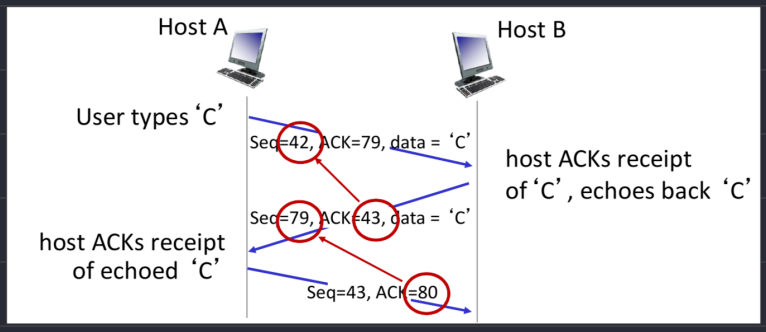


TCP sequence numbers, ACKs

Sequence numbers: Byte stream “number” of first byte in segment’s data

ACK numbers: seq # of next byte expected from other side (0 or 1)

ACK bit: 使否讀取 ACK nmuber field 並更新 sliding window



TCP round trip time, timeout

TCP 使用封包傳送的 RTT 來制定什麼時候 timeout

$$\text{EstimatedRTT} = (1-\alpha)*\text{EstimatedRTT} + \alpha*\text{SampleRTT}$$

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4*\text{DevRTT}$$ (Safety margin)

$$\text{DevRTT} = (1-\beta)*\text{DevRTT} + \beta*|\text{SampleRTT}-\text{EstimatedRTT}|$$

Fast re-transmittion

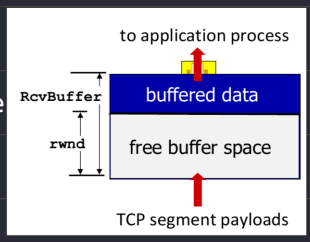
Original: 等到 timeout 在判定是否重送

Fast Re-transmit:發現收到 3 個重複的 ACK，判定是因為網路掉包而不是塞車，提前重送

Flow control

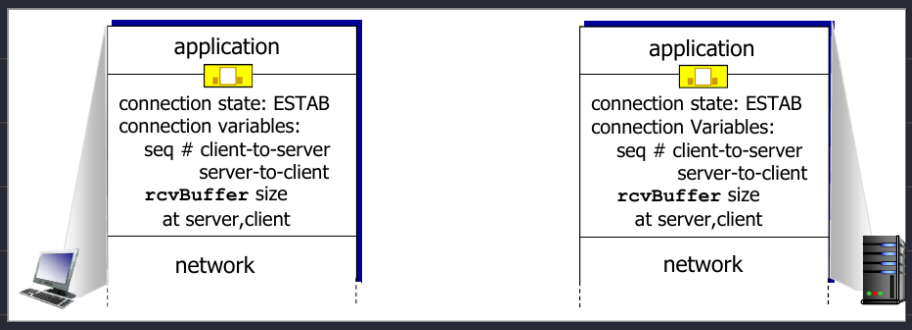
Receiver controls sender's 量 & 速度, so it won't overflow buffe

Receiver advertise "rwnd" in "receive window" field

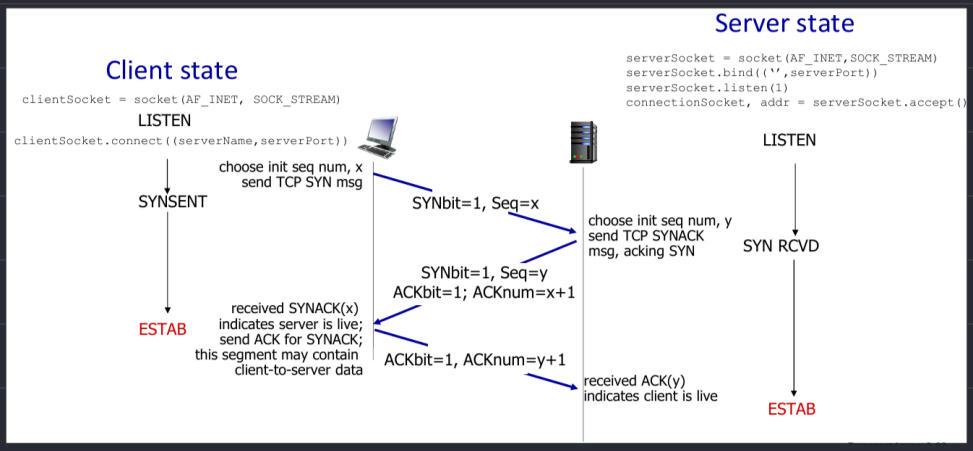


TCP connection management

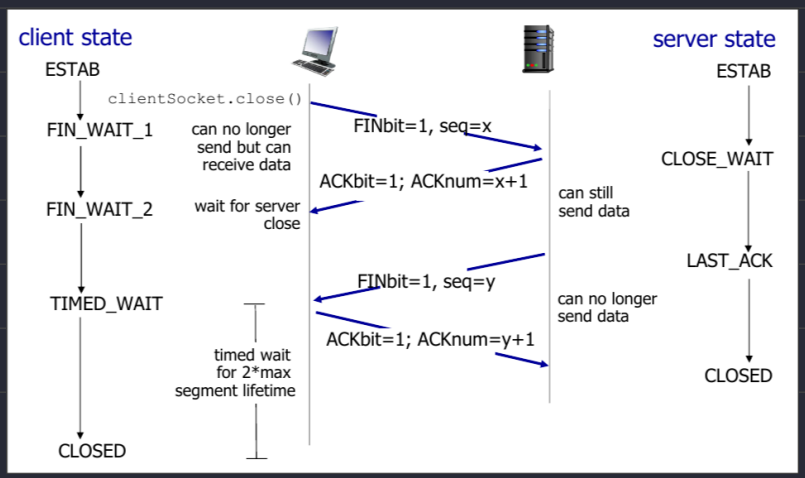
1. 建立主機之間連線　2. 同意連線中的參數（e.g. starting seq # ... ）



TCP 3-way handshake (TCP 三方交握)



Closing a TCP connection



Principles of congestion control

Sender sends too many data letting network can't handle

Manifestations:

- long delays (queueing in router buffers)
- packet loss (buffer overflow at routers)

Causes/costs of congestion:

Scenario 1

很多傳送端經過同一個 router把該 router的 queue幾乎佔滿，造成可觀的延遲

Scenario 2

Router 已經因為輸入封包過多開始 drop 了，但是傳送端因timeout還繼續堆封包

Scenario 3

假設一個封包經過 Router A~C 抵達 Router D，並在這裡丟包，那方才 Router A~C 的處理直接沒用了

Approaches towards congestion control

End-end: 網路層不參與流量狀態回報，只有終端設備觀察 loss, delay 執行

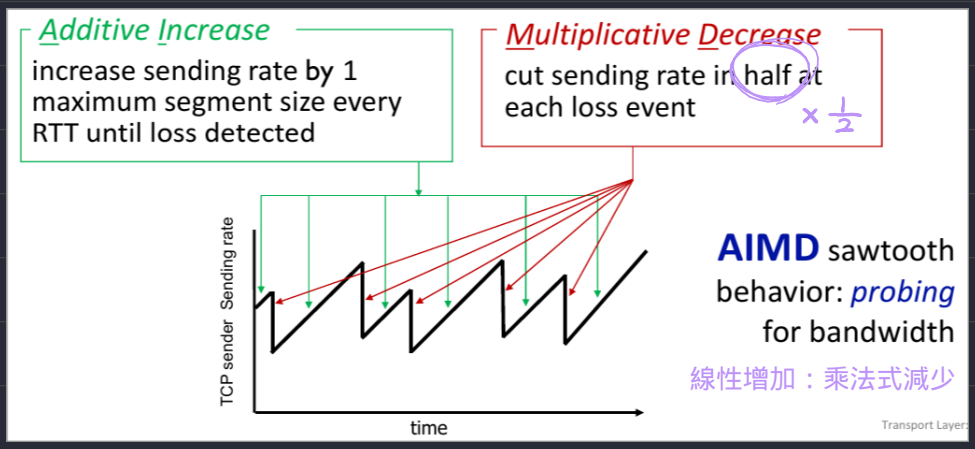
Network-assisted: routers 提供目前網路的狀態（目前不是所有router都支援）

TCP congestion control

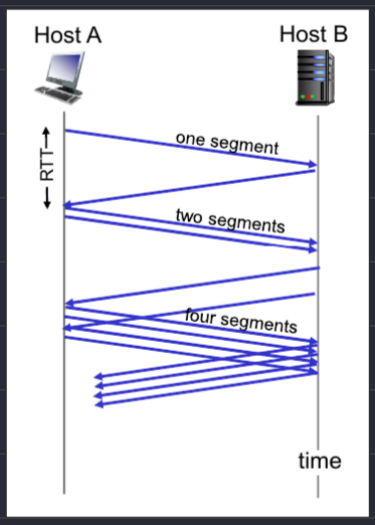
AIMD

Trasmitter can send min(recieving-window, congestio-window) data at the same time

Sender will increase the sending rate until packet loss occurs, and decrease rate



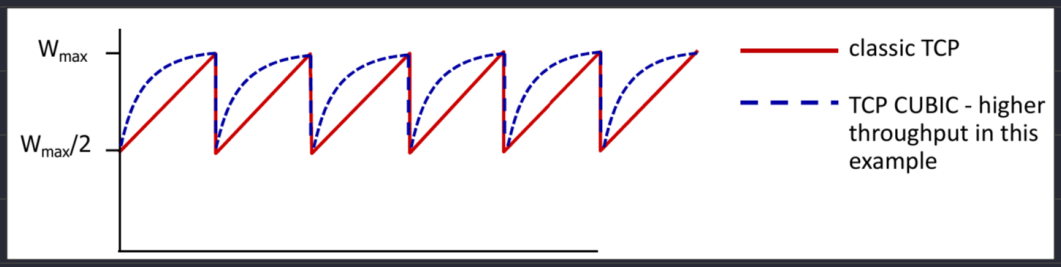
TCP slow start



variable ssthresh = 1/2 of cwnd just before loss event

after cwnd hit ssthresh, the increment mode will switch to linear (from *2 to +1)

TCP CUBIC (Briefly)



W_max: sending rate at which congestion loss was detected

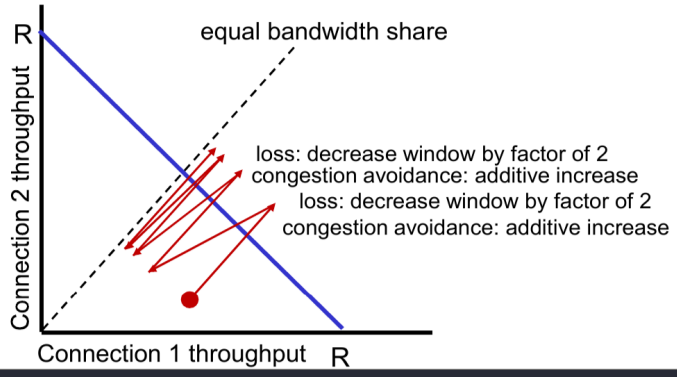
Congestion state of bottleneck link probably (?) hasn't changed much

After cutting rate/window in half on loss, initially ramp to to W max faster, but then approach W max more slowly

Q: is TCP Fair?

Example: two competing TCP sessions:

- additive increase gives slope of 1, as throughout increases
- multiplicative decrease decreases throughput proportionally



Is TCP fair?

A: Yes, under idealized assumptions:

- same RTT
- fixed number of sessions only in congestion avoidance

Evolution of transport-layer (Briefly)